

SQLite Online: conhecendo consultas

Mas o que é SQL ?

É uma linguagem *de consulta* que usa **comandos**, **cláusulas**, **expressões**, **predicados** para construir uma **query**.

Aula 01 - Importando a base de Dados

As importações foram feitas no SQLite Online. Aqui, eu estou usando o VS Code com a extensão SQLite.

As tabelas do curso são `tabela_fornecedores.csv` `tabela_pedidos.csv`. Vou carregá-las em um banco local com:

```
import pandas as pd
import sqlite3

# conexão ao banco local
conn = sqlite3.connect("banco_desafios.db")

df_clientes = pd.read_csv("tabela+clientes+desafio+aula+1+.csv", parse_dates=True)
df_produtos = pd.read_csv("tabela+produtos+desafio+aula+1.csv")
df_vendas = pd.read_csv("tabela+vendas+desafio+aula+1.csv")

# Adicionando ao banco.
df_clientes.to_sql("clientes", conn, if_exists="replace", index=False)
df_vendas.to_sql("vendas", conn, if_exists="replace", index=False)
df_produtos.to_sql("produtos", conn, if_exists="replace", index=False)
```

Criando 3 tabelas nomeadas: `clientes`, `vendas` e `produtos`.

Posso vê-las com em um arquivo `show_tables.sql` com:

```
.tables
```

Além disso, análogo ao método `.info()` de um DataFrame Pandas, podemos usar o comando nativo do SQLite PRAGMA:

```
PRAGMA table_info(nome_da_sua_tabela);
```

Aula 02 - Criando tabelas

2.1 Criando a tabela clientes

```
CREATE TABLE tabelaclientes (
  ID_Cliente INT PRIMARY KEY,
  Nome_Cliente VARCHAR (250),
  Informacoes_de_Contato VARCHAR (250)
);
```

Tipos de Dados 1 - Texto (String):

- **CHAR:** Armazena strings de tamanho fixo. Usado quando os valores têm um comprimento constante.
- **VARCHAR:** Armazena strings de tamanho variável. Adequado para valores com comprimentos variáveis.
- **TEXT (TEXT):** Armazena strings muito longas, como documentos ou descrições.

2 - Numérico:

INTEGER (INT): Armazena números inteiros.

FLOAT: Armazena números de ponto flutuante, geralmente usados para valores com casas decimais.

NUMERIC (DECIMAL): Armazena números com uma precisão específica, geralmente usados em aplicações financeiras.

3 - Data e Hora:

- **DATE:** Armazena datas sem informações de horário.
- **TIME:** Armazena informações de horário.
- **TIMESTAMP:** Combina data e horário em um único tipo.

4 - Booleano: - **BOOLEAN (BOOL):** Armazena valores verdadeiros ou falsos.

5 - Binário:

- **BLOB (Binary Large Object):** Armazena dados binários, como imagens, vídeos ou arquivos.
- **BIT:** Armazena valores binários, como 0 ou 1.

2.2 Alterando e excluindo tabelas

```
ALTER TABLE tabelaclientes ADD Endereco_Cliente VARCHAR (250);
DROP TABLE tabelaclientes
```

Comando DROP

```
DROP tipo_do_objeto nome_do_objeto;
```

Utilizado para **excluir** um objeto do banco de dados. Esse objeto pode ser: - índices
- tabelas - bancos de dados - usuários - funções - esquemas

Comando ALTER Age em *colunas* ou *tabelas*, alterando as suas estruturas. - Adicionar, modificar ou excluir colunas em uma tabela existente.

- Adicionar ou liberar **restrições** em tabelas.

2.3 PRIMARY KEY

Exemplos de Primary Key:

- Em uma tabela de “Clientes”, o número de identificação de cada cliente pode ser a chave primária.
- Em uma tabela de “Pedidos”, um número de pedido exclusivo pode servir como a chave primária.
- Em uma tabela de “Funcionários”, o número de identificação de cada funcionário pode ser a chave primária.

INT PRIMARY KEY vs INTEGER PRIMARY KEY

No SQLite, para colunas normais (que não são chaves primárias), não há diferença. Ambos recebem a mesma afinidade de tipo (`INTEGER`) e armazenam os dados exatamente da mesma maneira.

No entanto, há uma diferença crucial quando você define uma Chave Primária (Primary Key).

INTEGER PRIMARY KEY: No SQLite, essa declaração exata faz com que a coluna se torne um alias para o rowid (um número de identificação interno, exclusivo e oculto que o SQLite cria para cada linha). Como resultado, essa coluna se tornará autoincremental. Se você inserir um registro sem especificar o ID, o SQLite gerará o próximo número automaticamente.

INT PRIMARY KEY: Se você escrever `INT` em vez de `INTEGER`, o *SQLite não conectará essa coluna ao rowid*. Ela será apenas uma coluna normal com uma restrição de chave primária. O banco não vai gerar os números automaticamente, e você será obrigado a fornecer um ID manualmente a cada inserção.

Resumo da regra prática no SQLite: Para chaves primárias de identificação (como `id_projeto`), use sempre `INTEGER PRIMARY KEY`. Para outras colunas numéricas comuns, você pode usar `INT` ou `INTEGER` livremente.

2.4 FOREIGN KEY

É responsável por *relacionar* tabelas.

Importante: A chave estrangeira sempre se relacionará com uma chave primária de outra coluna. Na tabela de produtos que criamos, podemos analisar que a chave estrangeira, tanto da coluna `Categoria` quanto da coluna `Fornecedor`, está ligada às chaves primárias dessas tabelas, que são as colunas `ID_Categoria` e `ID` das tabelas de categoria e de fornecedores.

Aula 03 - Inserindo dados

Vamos inserir os clientes na tabela de clientes `tabelaclientes`.

3.1 Inserindo uma linha - INSERT INTO

Usamos a combinação `INSERT INTO + VALUES` onde a ordem dos valores devem ser na mesma ordem das colunas definidas

3.2 Inserindo múltiplas linhas

Continua utilizando o `INSERT INTO` mas com várias linhas para o campo `VALUES`. Vide os arquivos `raw/03_*.csv`.

DELETE Na execução do desafio, eu inseri na ordem errada alguns registros e limpei as linhas tabela com

```
DELETE FROM tabelaclientes;
```

E aí depois inseri os clientes da maneira correta.

3.3 INSERT + SELECT

Podemos usar uma sintaxe mais geral combinando INSERT e SELECT, movimentando dados de uma tabela para outra. Em geral a sintaxe é:

```
INSERT INTO tabela_destino (coluna1, coluna2, ... )
SELECT coluna1, coluna2, ...
FROM tabela_origem
WHERE condição;
```

3.4 Desafio

A tabela de funcionários possui os seguintes campos:

ID (Chave Primária)
Nome (Texto)
Cargo (Texto)
Departamento (Texto)
Data de Contratação (Data)
Salário (Decimal)

Escreva uma instrução SQL INSERT para adicionar um novo funcionário à tabela. Use os seguintes dados de exemplo:

ID: 32D
Nome: João Silva
Cargo: Desenvolvedor de Software
Departamento: TI
Data de Contratação: 2023-10-24
Salário: 6000.00

Resposta

```
INSERT INTO tabela_funcionarios (
    ID,
    Nome,
    Cargo,
    Departamento,
    Data_de_Contratacao,
    Salario
) VALUES
(32D, 'João Silva', 'Desenvolvedor de Software', 'TI', '2023-10-24', 6000.00)
```

Aula 04 - Consultando os dados

Os filtros em SQL quase sempre são acompanhados de cláusulas SELECT + WHERE.

Operadores de Comparação

- maior que (>)
- menor que (<)
- maior igual que (>=)
- menor igual que (<=)
- igual (=)
- diferente (<>) ou (!=)

Exemplo

```
-- Exemplo: Produtos que custam 50 ou mais, mas não custam exatamente 100
SELECT nome_produto, preco
FROM produtos
WHERE preco >= 50 AND preco <> 100;
```

Operadores lógicos:

Estes são essenciais para construir as condições de filtro (WHERE e HAVING). Eles avaliam se as condições são verdadeiras para retornar as linhas.

- O AND exige que ambas sejam verdadeiras
- OR exige que apenas uma seja verdadeira.
- IN substitui múltiplos OR verificando se um valor corresponde a qualquer valor dentro de uma lista especificada.
- BETWEEN: Filtra valores dentro de um intervalo (inclusive os valores limites). Funciona para números, textos e datas.

LIKE: Realiza buscas por padrões em textos usando curingas: - % (Representa zero, um ou vários caracteres)

- _ (Representa exatamente um caractere)
- IS NULL / IS NOT NULL: **A única forma correta de testar se um campo está vazio ou preenchido.**
- NOT não compara dois valores diretamente, ele simplesmente pega uma condição que retornaria Verdadeiro (TRUE) e a transforma em Falso (FALSE), e vice-versa.
 - Uso principal: Negar operadores avançados (IN, BETWEEN, LIKE, EXISTS, IS NULL) ou reverter o resultado de blocos inteiros de condições.

Ordenando com ORDER BY

O ORDER BY sempre aparece como a última cláusula estrutural de uma consulta (após o WHERE e o HAVING, se existirem). Ele utiliza duas palavras-chave principais para definir a direção da ordenação:

ASC (Ascendente): Do menor para o maior (A-Z, 0-9, datas mais antigas para as mais recentes). **É o comportamento padrão do SQL.**

DESC (Descendente): Do maior para o menor (Z-A, 9-0, datas mais recentes para as mais antigas).

Apelidos com AS

O comando **AS** (criador de Aliases ou apelidos) é utilizado para dar um nome temporário a uma coluna ou tabela durante a execução de uma consulta. Ele serve para tornar o resultado final mais legível ou para simplificar a escrita de códigos longos, sem alterar de forma definitiva a estrutura do banco de dados.

O banco de dados executa a consulta em uma ordem específica. O **SELECT** (onde o Alias é criado) é uma das últimas coisas a ser processada. Por causa disso, o Alias não existe no momento em que o banco está filtrando ou agrupando os dados.

Apelidos de Colunas

-- Renomeando colunas existentes para uma leitura melhor

```
SELECT nm_cli AS cliente, dt_nasc AS data_nascimento
FROM tb_clientes;
```

-- Dando nome a uma coluna calculada

```
SELECT
    nome_produto,
    preco,
    (preco * 0.90) AS preco_com_desconto
FROM produtos;
```

Apelidos de Tabelas

-- Sem Alias: Código longo e repetitivo

```
SELECT clientes.nome, pedidos.data_pedido, pedidos.valor_total
FROM clientes
JOIN pedidos ON clientes.id_cliente = pedidos.id_cliente;
```

-- Com Alias (c e p): Código limpo e fácil de ler

```
SELECT c.nome, p.data_pedido, p.valor_total
FROM clientes c
JOIN pedidos p ON c.id_cliente = p.id_cliente;
```

Aula 05 - Atualizando e excluindo os dados - DELETE e UPDATE

Os comandos **UPDATE** e **DELETE** pertencem ao subconjunto do SQL chamado DML (Data Manipulation Language). Diferente do **SELECT**, que apenas lê informações, esses dois comandos alteram o estado do banco de dados de forma permanente. O **UPDATE** é usado para alterar os valores de uma ou mais colunas em registros (linhas) que já existem na tabela.

UPDATE

```
UPDATE nome_da_tabela
SET coluna1 = novo_valor, coluna2 = novo_valor
WHERE condicao;
```

Para alterar vários campos ao mesmo tempo, basta separar as atribuições da cláusula SET por vírgula.

```
-- Atualiza o status e a data de entrega de um pedido específico
UPDATE pedidos
SET status_pedido = 'Entregue', data_entrega = '2026-09-29'
WHERE id_pedido = 1045;
```

DELETE

O DELETE é usado para **apagar linhas** inteiras de uma tabela. É importante notar que **você não pode apagar “apenas uma coluna” com o DELETE** (para limpar o valor de uma coluna específica, use o UPDATE e defina o valor como NULL).

Boas práticas

- Use um SELECT de segurança

```
-- Passo 1: Valide se o filtro retorna exatamente as linhas desejadas
SELECT * FROM clientes WHERE id_cliente IN (12, 45, 88);

-- Passo 2: Se o resultado estiver correto, troque o SELECT pelo DELETE
DELETE FROM clientes WHERE id_cliente IN (12, 45, 88);
```